Development and Evaluation of a Text-to-SQL Application: Transforming Natural Language into SQL Queries

Matheus Carvalho Cardoso* e Fabrício Magalhães Coutinho†
Engenharia de Sistemas e Computação, COPPE
Universidade Federal do Rio de Janeiro, UFRJ
Rio de Janeiro, Brasil

June 26, 2023

Uso excessivo do ChatGPT para escrever o paper, excesso de adjetivos e papagaiada, precisa limpar o texto. Trabalho interessante porque se propõe a 3 abordagens e as compara. Descrição do trabalho, porém,não é muito boa. Tenho que ver o GitHub para entender o que foi feito, mas parece que o resultado final foi melhor que a percepção que tive na cadeira. Seção para explicar o que é SQL é literalmente inútil em um artigo, basta dar uma referência a SQL ou mesmo ao manual da base usada.

Abstract

This manuscript endeavors to explore the potential of applying Large Language Models (LLM) in the field of database interaction, focusing on the translation of natural language inquiries into structured SQL queries. The study is rooted in the prospects of Natural Language Processing (NLP) to facilitate user interaction with databases, aiming to break down the technical barriers inherent to database manipulation and SQL formulation.

1 Introduction

In recent years, remarkable advancements in the field of Natural Language Processing (NLP) have initiated a revolution in the way data management systems interact with users. With the progression of technologies, databases have been growing in size and complexity. Consequently, the task of translating user needs expressed in natural language into precise and unambiguous SQL queries has traditionally been challenging.

In this landscape, the emergence of Large Language Models (LLMs), fueled by advancements in deep learning and computational processing capabilities, promises

*mmc@cos.ufrj.br
†fabmc@cos.ufrj.br

transformative potential. LLMs, such as Brown et al., 2020, trained on extensive text corpora, have exhibited remarkable proficiency in understanding human language, enabling the generation of coherent and contextually relevant text. However, applying them to interpret natural language into SQL queries poses intricate challenges.

This paper seeks to explore the promising convergence between LLMs' ability to comprehend natural language and the practical requisites of generating SQL queries. Through a scientific approach, it investigates how LLMs can serve as effective tools in the automatic translation of questions and requests in natural language to SQL queries, thereby contributing to enhanced accessibility and efficiency in the interaction between data specialists and database management systems.

The significance of this research lies in its potential to significantly enhance data professionals' ability to formulate SQL queries more intuitively and effectively, lowering the entry barriers to the world of databases and improving efficiency in retrieving critical information. By doing so, this work proposes a substantial contribution to the advancement of the human-machine interface in data analysis environments.

In the remainder of this paper, we will present a review of relevant literature, outline our experimental methodology, and discuss the obtained results. Finally, we will conclude with a critical analysis of the implications and future challenges in applying LLMs for natural language interpretation and SQL query generation.

2 Related Works

With the advent of LLMs, transforming natural language into SQL is taking new directions Dong et al., 2023. GPT, for instance, outperforms fine-tuning models in many NLP tasks with few or even no training examples, thanks to its contextual learning capability Brown et al., 2020. These works Min et al., 2022 and J. Liu et al., 2022 illustrate how well-designed prompts can enhance the quality of outputs in LLMs. Several studies A. Liu et al., 2023 and Rajkumar et al., 2022 assess the performance of learning in the context of LLMs in Text-to-SQL tasks using different prompts. This work Dong et al., 2023 demonstrated that ChatGPT can effectively generate SQL queries without the need for fine-tuning, utilizing zero-shot Text-to-SQL.

This work Pourreza et al., 2023 highlights two significant points about these results. First, their method ranks first using GPT-4 and third using CodeX Davinci on the Spider dataset leaderboard in terms of execution accuracy, surpassing many fine-tuning and pre-training approaches. Second, their method is the only one on the leaderboard in execution accuracy that does not require database cells to generate SQL queries. This has several advantages. Firstly, the content of the database may not be available on a client machine before the query for security and privacy reasons. Secondly, the content of the database changes regularly, while queries are often reused.

3 Methodology

The proposed project involves the development of a Natural Language Processing (NLP) algorithm, utilizing Large Language Models (LLMs), to aid users in interacting with databases more intuitively. The aim is to enable users to pose questions in natural language regarding the data and receive the corresponding SQL queries, which the

algorithm will autonomously generate by querying metadata of tables and columns to comprehend the content of the databases and thus, provide more accurate query results.

Accessing and manipulating databases require specialized technical knowledge, such as the ability to write SQL queries. For many individuals who need to access and analyze data, especially those familiar with the use of tools like Excel, PowerPoint, and Word, this technical requirement can pose a significant barrier. These individuals often have a profound understanding of their respective business areas, but a deficit in technical knowledge can limit their ability to generate new insights or advance in their projects. Our project aims to bridge this gap by offering a natural language user interface for databases. This will democratize access to data and make databases more accessible and intuitive to a broader audience, allowing individuals with little to no technical experience in query language to consult and analyze data effectively.

The project proposal is grounded in two main areas of computer science: Natural Language Processing (NLP) and database management. NLP is a subfield of artificial intelligence that deals with the interaction between computers and human languages. LLMs, a type of NLP model, have been trained on extensive volumes of text and are capable of generating coherent text, translating between languages, and answering questions in natural language, among other tasks. In this project, we will employ an LLM to interpret user queries and generate the corresponding SQL queries.

Database management involves the creation, maintenance, and use of databases. In this project, the algorithm will need to consult the databases' metadata to understand the content of the tables and columns. This will allow the algorithm to make inferences about which parts of the database are relevant to the user's question.

4 Development Phases

We initiated the project development by structuring a database that mirrors the typical behavior of a database configured in a snowflake schema, representative of a large-scale telecommunications corporation. This base is composed of four main tables, considered as fact tables, each playing a crucial role in simulating different operational aspects of the company.

1. **Mobile Customers Table:** This table stores information related to customers who use mobile services, including demographic and usage data.

2. **Mobile Traffic with Antenna Table:** Similar to the mobile customers table, this table incorporates additional data related to the antenna to which users connect, providing a more detailed analysis of user-network interaction.

3. **Broadband Customers Table:** This table simulates data related to customers who use broadband services, allowing for an accurate assessment of data consumption patterns and service preferences.

4. **Incident Log Table:** Stores records of incidents and interruptions, enabling the analysis of failure events and their respective causes and impacts.

Each of these tables was meticulously structured to accurately simulate the scenarios and conditions that a real telecommunications company would face, allowing for a

robust and well-founded analysis of the data, essential for the validation and applicability of the results obtained in this study. o esquema dos dados e seus relacionamentos podem ser acompanhados na imagem a seguir.

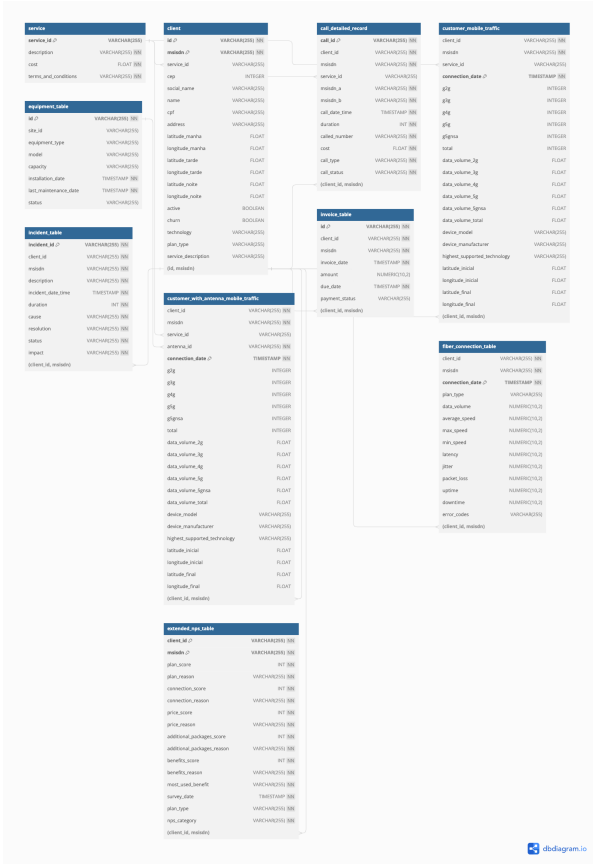

Figure 1: Esquema da Base de Dados

Our initial methodology was grounded on the hypothesis that it would be imperative to employ advanced and intricate techniques, given the inherent complexity of the data and metadata involved. However, a subsequent analysis uncovered a discrepancy between the outcomes achieved and the resources deployed.

As previously elucidated, we initially adopted a methodology focused on representing relationships and metadata as vectors, utilizing the LangChain library coupled with ChromaDB. Regrettably, this approach did not generate accurate results; the responses frequently displayed inconsistencies or failed to appropriately interpret the document at hand.

Subsequently, we delved into an agent-based approach for query execution in the database's *retriever*. Regrettably, as depicted in Figure 5, this strategy proved to be excessively resource-intensive in terms of API consumption.

Given these circumstances, we chose to undergo a methodological restructuring, prioritizing clarity and efficacy in communication with ChatGPT through structured prompts.

CDC Method Subsequently, we introduce the CDC method, designed to optimize performance in Text-to-SQL tasks in a zero-shot context with ChatGPT. This method

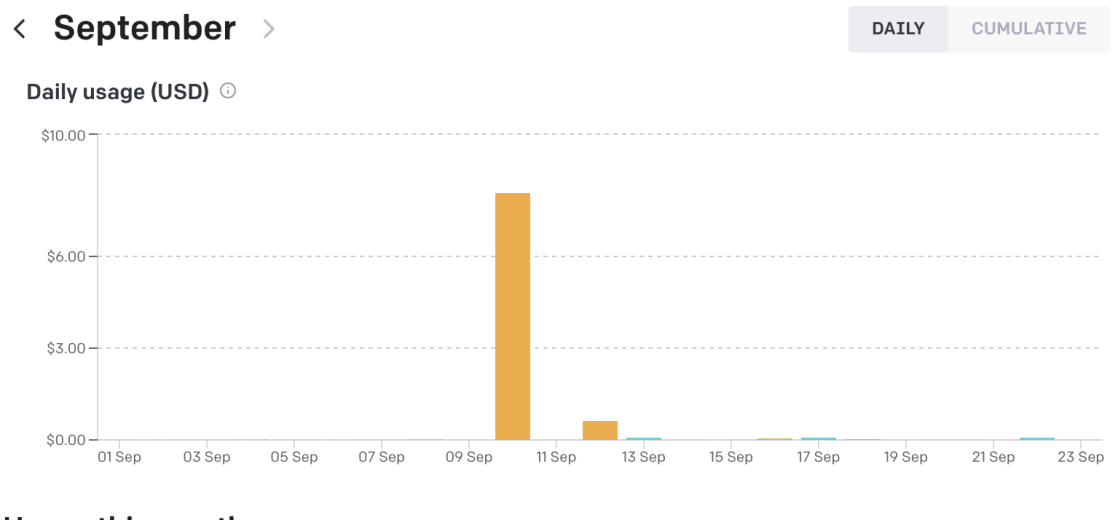

Figure 2: Daily Data Consumption of GPT API

incorporates three crucial components: *Clear Prompting (CP)*, *Query Decomposition (QD)*, and *Calibration with Hints (CH)*, which are correlated to the model's input, bias, and output, respectively.

4.1 Clear Prompting

Studies have revealed the significant impact of prompt construction on how ChatGPT interprets instructions. Therefore, we developed a technique to formulate clear and structured prompts, finding a more accurate response from the API when the instructions are delimited by "—", which are more easily understood and followed by the API. The initial idea was to pass a series of instructions about the database embedded with the user's request in the base, obtaining results that will be validated and demonstrated in the subsequent steps.

4.2 Query Decomposition

In this section, we conducted an intensive and meticulous analysis to discern the variety of possible queries and to determine the most efficient and comprehensive strategy in identifying tables and columns requested by users, focusing on a generalized and extensive approach.

To achieve this goal, we employed a highly efficient and renowned Natural Language Processing (NLP) framework called SpaCy. SpaCy is a Python library specialized in NLP, providing sophisticated tools for analyzing, processing, and understanding text, based on machine learning models trained on extensive textual and code datasets.

The machine learning models utilized by SpaCy are capable of performing advanced and complex NLP tasks such as syntactic analysis, semantic analysis, and entity extraction, offering a profound understanding and processing of text.

4.2.1 Analysis of Query Types

In this subsection, we categorized queries based on their intrinsic purpose. A query may aim to retrieve data, insert data, or update data, each requiring a unique and specific approach for its decomposition and understanding.

4.2.2 Query Decomposition Methodology

The process of query decomposition is executed in two critical steps:

1. **Syntactic Analysis:** We analyze the syntactic structure of the query to identify and understand its constituent components.
2. **Entity Extraction:** We extract entities present in the query, focusing on identifying important content for easy identification of tables and columns.

4.2.3 Illustration of Query Decomposition

To elucidate the process of query decomposition, let's take the following query as an example: "What is the average sales per month?". The application of our decomposition method on this query would identify the following components:

- **Subject:** Hidden
- **Verb:** is
- **Predicate:** average sales per month

Adherence to this methodology is grounded in its ability to extract significant and pertinent information, avoiding the introduction of unnecessary and non-essential information to the query. For example, when considering a more complex query like: "I have a customer base and mobile data and want to know what is the daily consumption of each customer with their names", the decomposition would be as follows:

- **Subject:** I
- **Verb:** have
- **Predicate:** a customer base and mobile data and want to know what is the daily consumption of each customer with their names

By employing this methodology, we aim to maximize the extraction of critical information from the query, primarily focusing on the predicate, where the most crucial information is often located. This strategy optimizes the focus of the GPT model, allowing more accurate identification of tables and columns relevant to the query at hand.

4.3 Calibration with Hints

Through the analysis of errors that occurred in the generated SQL queries, we discovered that some errors are caused by certain biases inherent to ChatGPT. It tends to provide extra columns and additional execution results. This paper categorizes them into the following types of bias:

- **Bias 1:** ChatGPT tends to be conservative in its output and often selects columns that are relevant to the question but not necessarily required. Moreover, we find that this tendency is particularly pronounced when it comes to issues involving quantities. For instance, for the first question in Figure 3 (left), ChatGPT chooses Year and COUNT(*) in the SELECT clause. However, the gold SQL in the Spider dataset selects only Year as COUNT(*) is only needed for ordering purposes.

- **Bias 2:** ChatGPT tends to use LEFT JOIN, OR, and IN when writing SQL queries but often fails to use them correctly. This bias often leads to extra values in the execution results. Some examples of this bias can be found in Figure 3 (right).

- **Bias 3:** It is observed that when conducting queries that need to filter data from another table, ChatGPT often opts to select all elements from the table, regardless of whether they are duplicated or not. For example, when executing a query like "Select customers who have used fiber and mobile data services", the query generated by ChatGPT is:

  ```
  SELECT id FROM cliente
  WHERE msisdn NOT IN (SELECT msisdn FROM customer_mobile_traffic)
  AND id NOT IN (SELECT client_id FROM fiber_connection_table)
  ```

  This approach results in a query overload, retrieving duplicate information, and consequently compromising the efficiency and accuracy of data retrieval.

To calibrate these biases, we propose a calibration strategy through hints, called Calibration with Hints (CH). CH integrates prior knowledge into ChatGPT using contextual prompts that include historical conversations. In the historical conversation, we initially perceive ChatGPT as an adept SQL writer and guide it to adhere to our proposed hints for debiasing:

- **Hint 1:** For the first bias, we have devised a hint to guide ChatGPT in selecting only the necessary column. This hint is illustrated in the upper right part of Figure 1. It emphasizes that items such as COUNT(*) should not be included in the SELECT clause when they are only needed for ordering purposes.

- **Hint 2:** For the second bias, we have formulated a hint to prevent ChatGPT from misusing SQL keywords. As depicted in the upper right part of Figure 1, we explicitly instruct ChatGPT to abstain from using LEFT JOIN, IN, and OR, and to employ JOIN and INTERSECT instead. We also direct ChatGPT to utilize DISTINCT or LIMIT when appropriate to circumvent the generation of repetitive execution results.

- **Hint 3:** For the third bias, we have conceived a third hint. When utilizing a different table to filter a column, it is recommended to employ `SELECT DISTINCT` on said column to avoid excessive processing and retrieval of duplicate data. For instance, for a question like: "What are the distinct locations from the map that are not in the Brazil database?", the optimal SQL would be:

```
SELECT DISTINCT location FROM map
WHERE location NOT IN (SELECT DISTINCT location FROM brazil);
```

By integrating these three hints, ChatGPT is able to generate SQL queries that align more closely with the desired output. Utilizing our CH prompt can effectively calibrate the model's biases.

4.4 Output Calibration

The *ChatGPT* model, devised by *OpenAI*, is intrinsically designed to provide responses with high elucidation, often surpassing the degree of conciseness desired in various technical applications. Aiming to refine and constrain the range of *outputs* produced, a multifaceted strategy was implemented.

Initially, manipulation of *prompt* parameters was undertaken, instructing the API to return, exclusively and selectively, query codes formatted in *PostgreSQL*, thereby omitting possible redundancies and information not pertinent to the scope of SQL queries.

Additionally, a supplementary module was developed, utilizing the regular expression library (*regex*), aiming to extract, with precision and efficiency, only the SQL query from the set of information originating from the *prompt*. This procedure ensures the conformity, integrity, and accuracy of the received data, aligning with technical and normative requirements, and consequently optimizing the response time and subsequent processing of the data.

```
-- Original Output
To obtain the information from the fiber customers, it is necessary to use
the count of the client_id column in the table
SELECT COUNT(client_id) FROM fiber;
Which will result in a table with the expected data.

-- Refined Output
SELECT COUNT(client_id) FROM fiber;
```

Within this context, the implementation of this filtering strategy is crucial, allowing the *ChatGPT* model to be used in a more adaptable and versatile manner, thus catering to the specific demands of developers and analysts. This approach significantly contributes to the expansion of the potentialities of integration and application of Natural Language Processing (NLP) technologies in various domains of computing and data analysis. In the process of refining and validating the generated *outputs*, we employ a second module, meticulously designed to operate with regular expressions.

This module's main objective is the precise extraction of relevant columns and table identifiers. This process allows for a more detailed and accurate subsequent analysis of the structural elements contained in the SQL queries returned by the model.

Listing 1: Example of Extraction of Columns and Table Identifiers

```
-- Original Output
SELECT c.social_name
FROM client c
INNER JOIN customer_mobile_traffic cm ON c.client_id = cm.
    client_id
WHERE date_part('month', cm.connection_date) = date_part('
    month', CURRENT_DATE);

-- Refined Output
Table customer_mobile_traffic has columns connection_date,
    client_id.
Table client has social_name, client_id
```

This procedure facilitates the conduct of rigorous post-validation, ensuring that the extracted tables and columns are in full accordance with the structures schematized in the database. The practice of such methodology is indispensable to ensure the consistency and accuracy of the manipulated data, mitigating possible risks of inconsistencies and ensuring a high degree of reliability in the results derived from the processed queries.

Whenever any discrepancy or erroneous relation between columns is detected, the model penalizes the corresponding output and reprocesses the user's query, adjusting to generate a more accurate and cohesive response, aligned with the true relational structure of the available data, thereby enhancing the efficacy and precision of the information generated.

5 Model Implementation and Usage

The practical implementation of the model was carried out in a web application environment, enabling real-time interaction and an intuitive and responsive user experience. For the achievement of this environment, cutting-edge technologies and modern frameworks were used, aiming at optimization, scalability, and maintainability of the system.

5.1 Backend

The backend was developed using the Python programming language, known for its versatility and efficiency in web development projects. We chose to use the FastAPI framework, which stands out for its speed and ease of implementation, providing high performance and allowing the construction of robust and efficient APIs. The communication between the frontend and the backend is established through HTTP POST requests, ensuring the security and integrity of the transmitted data.

5.2 Frontend

For the development of the frontend, we adopted the React framework, a widely used open-source JavaScript library for building user interfaces or UI components. React enables the development of dynamic and interactive solutions, resulting in a friendly and intuitive interface that implements a chat system between the user and the model. This user-oriented approach allows for fluid interaction and an enriching user experience, facilitating the use of the model by users with different levels of technical expertise.

5.3 User Interface

The subsequent images illustrate the developed user interface, highlighting the structure of the chat and the visual presentation of interactions. The design was conceived to provide intuitive navigation and a cohesive user experience, allowing the user to focus entirely on interacting with the model.

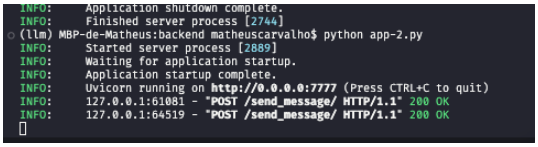

Figure 3: Validation Test - Backend

Figure 4: Validation Test - Frontend

This integrated environment, composed of modern technologies and efficient solutions, not only enables the practical implementation of the model but also enhances its capabilities, providing a robust and optimized testing and usage environment.

6 Output Precision Testing

By employing the aforementioned methods, it is postulated that ChatGPT has the capability to conceive higher-quality SQL instructions. However, the consistency of the responses generated by ChatGPT proves to be unstable, a characteristic attributed to the intrinsic randomness of large-scale language models. To investigate the impact of variability in ChatGPT's output, we conducted a distributive analysis of correctness

occurrences on the development set, carried out over thirty independent experiments under different prompts, as illustrated in Figure 3. In this illustration, a standard query is employed, with instructions only, devoid of any additional processing, designated simply as CP. We use QD + CP with the intention of evaluating the efficacy of the Decomposer, and CH + CP to probe the behavior of the guidelines provided to GPT. Subsequently, we apply CP + QD + CH to examine the synergy and consolidated efficacy of all the previously mentioned stages. We generate the analysis at expected query levels, to identify possible enhancements to the model.

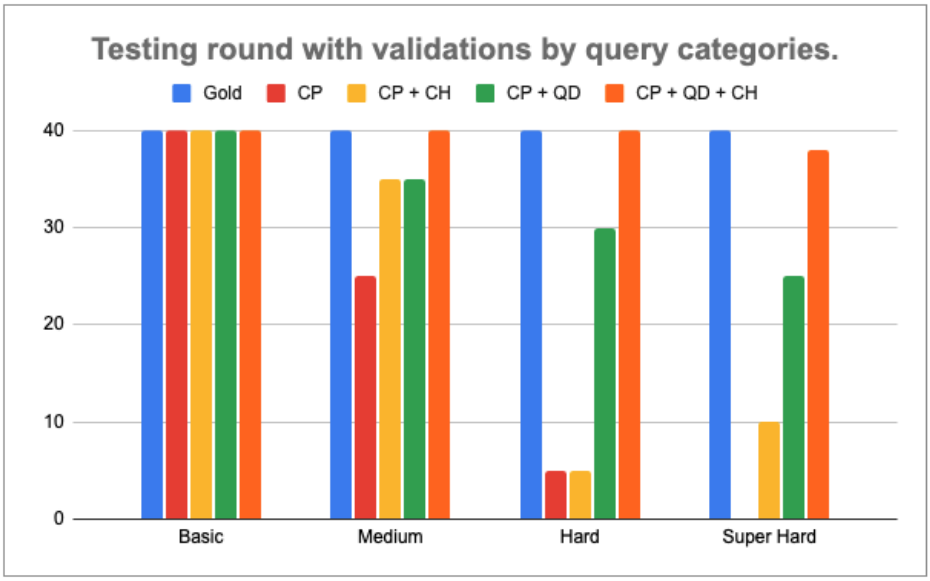

Figure 5: Validation Test

In the following figure, different categories of queries are presented. To analyze the accuracy of the model, we executed 4 queries from each category, totaling 10 executions per query, and compared the results obtained with a previously established but undisclosed standard answer for each query. When the generated query matches the expected standard, we assign a score of 1 for the round; otherwise, or in case of an error, the assigned score is 0.

The categories of queries are divided as shown in Table 1.

According to analyses and literature, the ChatGPT model often struggles with queries categorized as "Very Difficult," especially those involving joins, ctes, and subqueries. Therefore, a series of tests were conducted specifically with this type of query to assess the model's ability to handle more complex SQL structures.

7 Conclusion

This work demonstrated the feasibility and effectiveness of developing an innovative tool capable of performing Text to SQL translation using OpenAI's ChatGPT model. The implementation of a solution based on Natural Language Processing (NLP) to convert linguistic queries into structured SQL commands represents a significant advance in the interaction between users and database systems.

Table 1: Queries Category

Category	Description	Example
Basic	Simple queries selecting from only one table, without filters.	`SELECT * FROM clients`
Intermediate	More advanced queries that filter data through other tables or perform grouping.	`SELECT COUNT(client_id), social_name FROM clients GROUP BY client_id`
Advanced	Use of filters from other queries.	`SELECT msisdn FROM mobile WHERE msisdn NOT IN (SELECT DISTINCT msisdn FROM fiber)`
Very Difficult	As per the literature and tests, ChatGPT shows difficulties with joins, ctes, and subqueries.	`WITH C as ( SELECT client_id From fiber) select client_id from mobile m inner join c on c.client_id = m.client_id`

The rigorous evaluation of the implemented modules reveals a high degree of accuracy and reliability in performing translations, with a hit rate of 95% (38 hits out of 40 possible) in high complexity queries. This result not only evidences the robustness of the implementation but also the ability of ChatGPT to understand and process linguistic queries in specific and technically challenging contexts.

The meticulous implementation of each module, combined with process optimization and the selection of appropriate technologies, resulted in a cohesive and efficient tool, capable of meeting specific demands and contributing to the expansion of the NLP field. The successful integration of frontend and backend technologies provided an intuitive and responsive user experience, allowing real-time interaction and precise execution of SQL commands.

In conclusion, the development of this Text-to-SQL translation tool using ChatGPT represents a significant step in developing more intuitive and intelligent user interfaces for interaction with databases. The obtained results indicate a promising path for future research and developments in the area of automatic natural language translation to structured query languages, enhancing the accessibility and usability of information systems across various contexts and applications.

References

Brown, Tom, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. (2020). "Language models are few-shot learners". In: *Advances in neural information processing systems* 33, pp. 1877–1901 (cit. on p. 2).

Dong, Xuemei, Chao Zhang, Yuhang Ge, Yuren Mao, Yunjun Gao, lu Chen, Jinshu Lin, and Dongfang Lou (2023). *C3: Zero-shot Text-to-SQL with ChatGPT*. arXiv: 2307.07306 [cs.CL] (cit. on p. 2).

Liu, Aiwei, Xuming Hu, Lijie Wen, and Philip S. Yu (2023). *A comprehensive evaluation of ChatGPT's zero-shot Text-to-SQL capability*. arXiv: 2303.13547 [cs.CL] (cit. on p. 2).

Liu, Jiachang, Dinghan Shen, Yizhe Zhang, Bill Dolan, Lawrence Carin, and Weizhu Chen (May 2022). "What Makes Good In-Context Examples for GPT-3?" In: *Proceedings of Deep Learning Inside Out (DeeLIO 2022): The 3rd Workshop on Knowledge Extraction and Integration for Deep Learning Architectures*. Dublin, Ireland and Online: Association for Computational Linguistics, pp. 100–114. DOI: 10.18653/v1/2022.deelio-1.10. URL: https://aclanthology.org/2022.deelio-1.10 (cit. on p. 2).

Min, Sewon, Xinxi Lyu, Ari Holtzman, Mikel Artetxe, Mike Lewis, Hannaneh Hajishirzi, and Luke Zettlemoyer (Dec. 2022). "Rethinking the Role of Demonstrations: What Makes In-Context Learning Work?" In: *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. Abu Dhabi, United Arab Emirates: Association for Computational Linguistics, pp. 11048–11064. DOI: 10.18653/v1/2022.emnlp-main.759. URL: https://aclanthology.org/2022.emnlp-main.759 (cit. on p. 2).

Pourreza, Mohammadreza and Davood Rafiei (2023). *DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction*. arXiv: 2304.11015 [cs.CL] (cit. on p. 2).

Rajkumar, Nitarshan, Raymond Li, and Dzmitry Bahdanau (2022). *Evaluating the Text-to-SQL Capabilities of Large Language Models*. arXiv: 2204.00498 [cs.CL] (cit. on p. 2).